Project Synopsis

AI-Powered AUTOSAR High-Level Design Analysis Assistant

1. Project Identification & Scope

Project Title: AI-Powered AUTOSAR High-Level Design (HLD) Document Analysis Assistant

Domain: Automotive Systems Engineering, AUTOSAR Classic & Adaptive Architecture, Local RAG Systems

Target Environment: Fully Local, On-Premise Execution (Zero Confidential Data Egress)

Target Users: Automotive System Architects, Software Integration Engineers, Safety Auditors

2. Problem Statement & Motivation

Modern automotive electronic control units (ECUs) govern complex distributed functions spanning powertrain, chassis, and ADAS domains. High-Level Design (HLD) documents routinely exceed hundreds of pages, describing intricate Software Components (SWCs), Virtual Functional Bus (VFB) port interfaces, cyclic runnables, and hundreds of signals.

Manual engineering review of these specifications is labor-intensive, error-prone, and prone to oversight regarding missing interface definitions, contradictory signal data types, or undeclared dependencies. Existing cloud-based generative AI systems cannot be deployed due to strict confidentiality of proprietary OEM architectures.

3. Proposed Solution & Architecture

This project implements an end-to-end, locally hosted AI assistant that automates document ingestion, semantic chunking, dense vector retrieval, and grounded conversational question answering.

Key Architectural Capabilities:

• Local RAG Knowledge Base: PyMuPDF context-aware chunking with BAAI/bge-small-en-v1.5 embeddings stored in persistent ChromaDB.

• Grounded Local LLM: Ollama-powered Qwen model with strict anti-hallucination guardrails and mandatory page-level citations.

• Architecture Entity Extractor: Hybrid rule-based regex and Pydantic-validated extraction of SWCs, Interfaces, Signals, and Dependencies.

• Interactive Visualization: Dynamic directed dependency graph rendered using NetworkX and Plotly.

• Automated Consistency Audits: Deterministic checks for duplicate SWC names, undefined references, and interface naming anomalies.

• Multi-Format Reporting: Downloadable JSON and CSV engineering reports directly from the Streamlit UI.

4. Technology Stack Summary

Programming Language: Python 3.12 | UI Framework: Streamlit | PDF Engine: PyMuPDF (fitz) | Vector Store: ChromaDB | Embeddings: BAAI/bge-small-en-v1.5 | LLM Inference: Local Ollama (Qwen 3.5/2.5) | Visualization: NetworkX + Plotly | Data Validation: Pydantic v2 | Reporting: Pandas, JSON, CSV